Week 9

Experiment 1: Creating and Using a User-Defined Package

Aim: To create a user-defined package and access its class from another Java program.

Tasks:

- Create a package named mypackage.
- Create a class Student inside the package.
- Define student details and a method to display them.
- Create another class outside the package.
- Import and use the Student class.

Folder Structure:

```
C:\JavaPrograms
|-- TestStudent.java
|-- mypackage
    |-- Student.java
```

Program 1: mypackage\Student.java

```
package mypackage;

public class Student {
    private int rollNo;
    private String name;
    private String course;

    public Student(int rollNo, String name, String course) {
        this.rollNo = rollNo;
        this.name = name;
        this.course = course;
    }

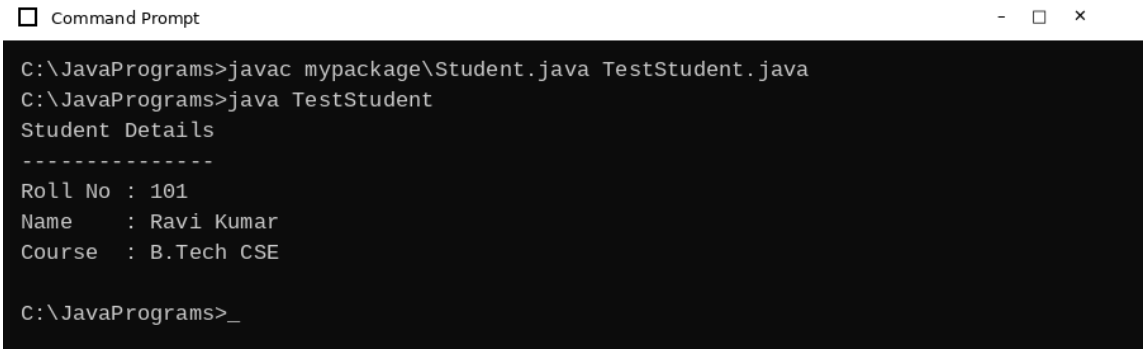
    public void display() {
        System.out.println("Roll No : " + rollNo);
        System.out.println("Name    : " + name);
        System.out.println("Course  : " + course);
    }
}
```

Program 2: TestStudent.java

```
import mypackage.Student;

public class TestStudent {
    public static void main(String[] args) {
        Student s = new Student(101, "Ravi Kumar", "B.Tech CSE");
        System.out.println("Student Details");
        System.out.println("---------------");
        s.display();
    }
}
```

Compilation and Output:



Experiment 2: Importing Packages

Aim: To demonstrate different ways of importing and accessing classes from a package.

Tasks:

- Create a package college.
- Create classes Student and Faculty.
- Import a specific class from the package.
- Demonstrate wildcard import using college.*.
- Access the classes from another program.

Folder Structure:

```
C:\JavaPrograms
|-- SpecificImport.java
|-- WildcardImport.java
|-- college
    |-- Student.java
    |-- Faculty.java
```

Program 1: college\Student.java

```
package college;

public class Student {
    private int id;
    private String name;

    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    public void display() {
        System.out.println("Student -> ID: " + id + ", Name: " + name);
    }
}
```

Program 2: college\Faculty.java

```
package college;

public class Faculty {
    private String name;
    private String department;

    public Faculty(String name, String department) {
        this.name = name;
        this.department = department;
    }

    public void display() {
        System.out.println("Faculty -> " + name + ", Dept: " + department);
    }
}
```

Program 3: SpecificImport.java

```
import college.Student;   // importing one specific class

public class SpecificImport {
```

```
    public static void main(String[] args) {
        System.out.println("--- Specific import ---");
        Student s = new Student(1, "Anil");
        s.display();

        // Faculty is not imported, so the fully qualified name is used
        college.Faculty f = new college.Faculty("Dr. Rao", "CSE");
        f.display();
    }
}
```

Program 4: WildcardImport.java

```
import college.*;   // importing all classes of the package

public class WildcardImport {
    public static void main(String[] args) {
        System.out.println("--- Wildcard import (college.*) ---");
        Student s = new Student(2, "Sita");
        Faculty f = new Faculty("Dr. Mehta", "ECE");
        s.display();
        f.display();
    }
}
```

Compilation and Output:



Experiment 3: Packages and Member Access

Aim: To demonstrate the accessibility of class members across packages using access modifiers.

Tasks:

- Create a class containing public, private, protected and default members.
- Access these members from the same class.
- Access these members from another class in the same package.
- Access these members from a class in a different package.

Folder Structure:

```
C:\JavaPrograms
|-- pack1
|   |-- Access.java
|   |-- SamePackage.java
|-- pack2
    |-- DifferentPackage.java
```

Program 1: pack1\Access.java

```
package pack1;

public class Access {
    public int pubVar = 10;
    private int priVar = 20;
    protected int proVar = 30;
    int defVar = 40;            // default (package-private)

    public void showAll() {
        System.out.println("Inside the same class:");
        System.out.println("public    : " + pubVar);
        System.out.println("private   : " + priVar);
        System.out.println("protected : " + proVar);
        System.out.println("default   : " + defVar);
    }
}
```

Program 2: pack1\SamePackage.java

```
package pack1;

public class SamePackage {
    public static void main(String[] args) {
        Access a = new Access();
        a.showAll();

        System.out.println("\nAnother class, same package:");
        System.out.println("public    : " + a.pubVar);
        // System.out.println(a.priVar);   // ERROR: private
        System.out.println("protected : " + a.proVar);
        System.out.println("default   : " + a.defVar);
    }
}
```

Program 3: pack2\DifferentPackage.java

```
package pack2;

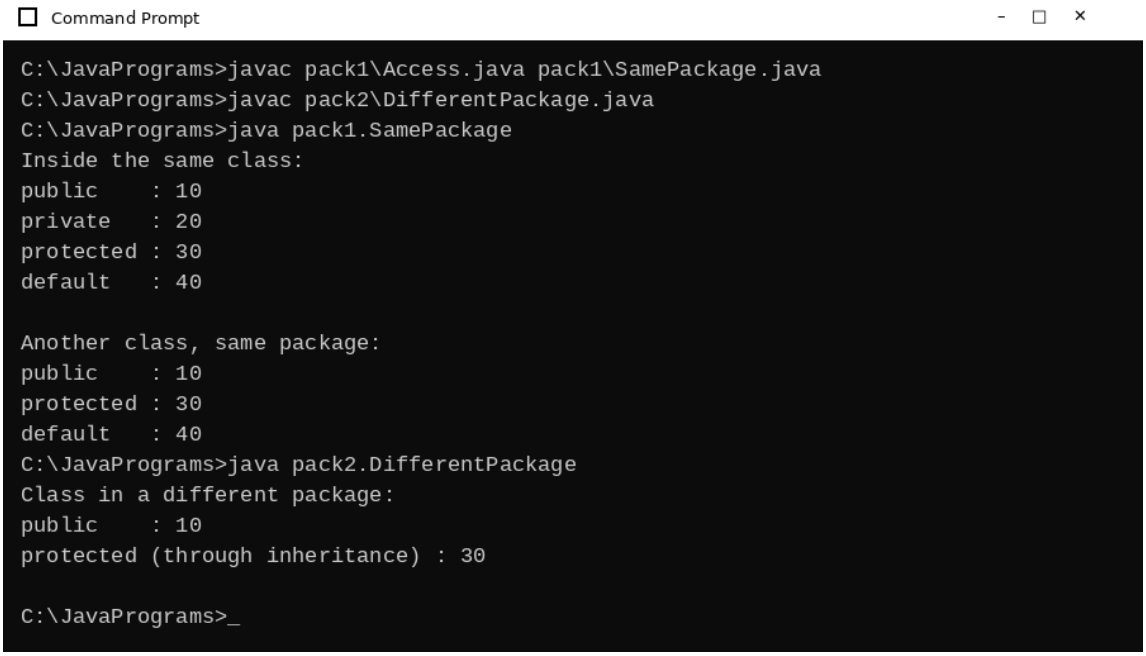
import pack1.Access;

public class DifferentPackage extends Access {
```

```
    public static void main(String[] args) {
        Access a = new Access();
        System.out.println("Class in a different package:");
        System.out.println("public    : " + a.pubVar);
        // a.priVar -> ERROR (private)
        // a.defVar -> ERROR (default: same package only)
        // a.proVar -> ERROR (protected: not through an Access reference)

        DifferentPackage d = new DifferentPackage();
        System.out.println("protected (through inheritance) : " + d.proVar);
    }
}
```

Compilation and Output:



Summary of Member Access:

Modifier	Same class	Same package	Different package
public	Yes	Yes	Yes
protected	Yes	Yes	Only through inheritance (subclass)
default	Yes	Yes	No
private	Yes	No	No

Experiment 4: Demonstrate CLASSPATH

Aim: To demonstrate how Java uses the CLASSPATH to locate user-defined packages and classes.

Tasks:

- Create a package utilities.
- Create a class Calculator.
- Compile the package.
- Set the CLASSPATH.
- Write another Java program that uses the Calculator class.
- Compile and execute the program using the configured CLASSPATH.

Folder Structure:

```
C:\JavaPrograms
|-- lib
|   |-- utilities
|       |-- Calculator.java
|-- app
    |-- UseCalculator.java
```

Program 1: lib\utilities\Calculator.java

```
package utilities;

public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }

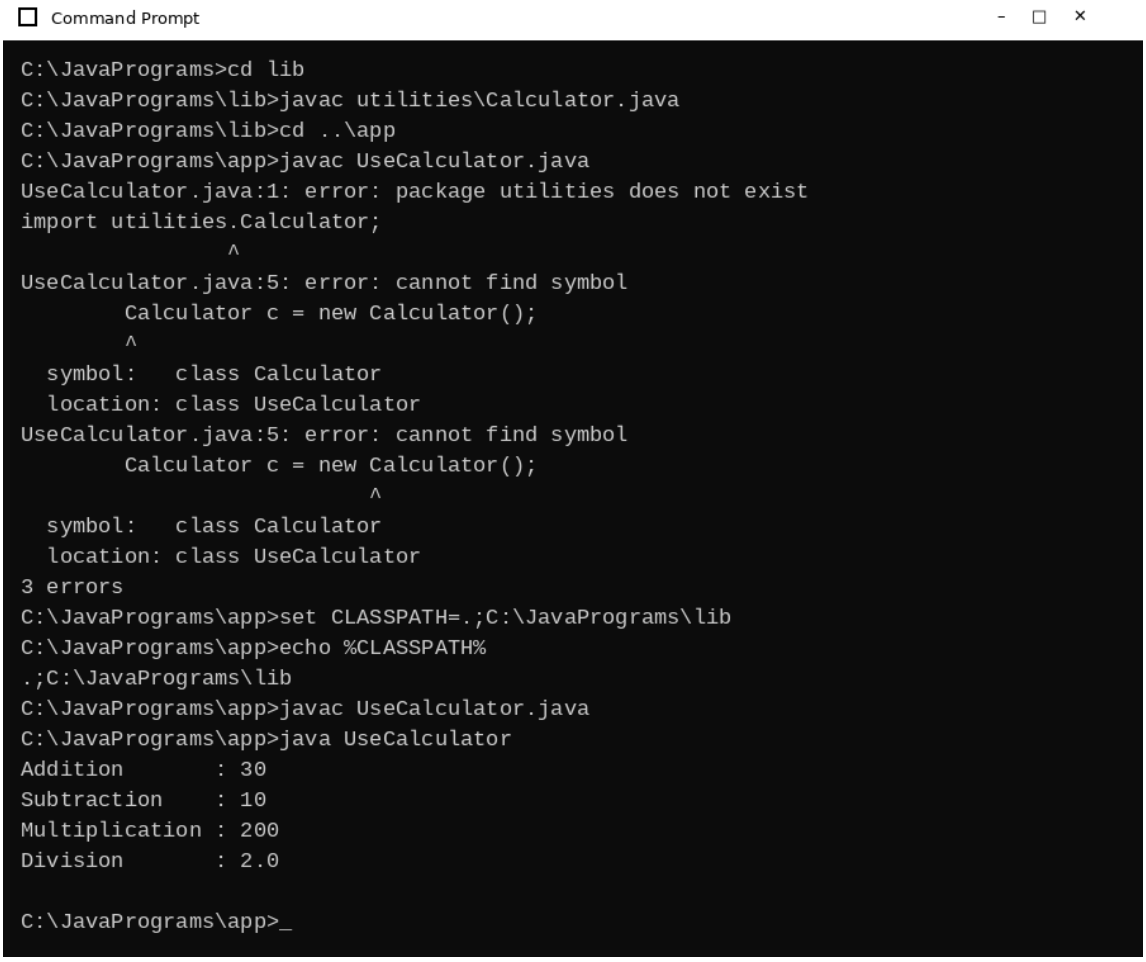
    public double divide(int a, int b) {
        return (double) a / b;
    }
}
```

Program 2: app\UseCalculator.java

```
import utilities.Calculator;

public class UseCalculator {
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println("Addition       : " + c.add(20, 10));
        System.out.println("Subtraction    : " + c.subtract(20, 10));
        System.out.println("Multiplication : " + c.multiply(20, 10));
        System.out.println("Division       : " + c.divide(20, 10));
    }
}
```

Compilation and Output:



Experiment 5: Package Containing Multiple Classes

Aim: To create and use a package containing multiple classes.

Tasks:

- Create a package named bank containing the classes Account, Customer and Transaction.
- Create a main class outside the package and access all three classes.

Folder Structure:

```
C:\JavaPrograms
|-- BankApp.java
|-- bank
    |-- Customer.java
    |-- Account.java
    |-- Transaction.java
```

Program 1: bank\Customer.java

```
package bank;

public class Customer {
    private int customerId;
    private String name;
    private String phone;

    public Customer(int customerId, String name, String phone) {
        this.customerId = customerId;
        this.name = name;
        this.phone = phone;
    }

    public void display() {
        System.out.println("Customer ID : " + customerId);
        System.out.println("Name        : " + name);
        System.out.println("Phone       : " + phone);
    }
}
```

Program 2: bank\Account.java

```
package bank;

public class Account {
    private long accountNo;
    private Customer customer;
    private double balance;

    public Account(long accountNo, Customer customer, double balance) {
        this.accountNo = accountNo;
        this.customer = customer;
        this.balance = balance;
    }

    public void deposit(double amount) {
        balance += amount;
    }

    public boolean withdraw(double amount) {
        if (amount > balance) {
            return false;
        }
        balance -= amount;
```

```
        return true;
    }

    public void display() {
        customer.display();
        System.out.println("Account No  : " + accountNo);
        System.out.printf("Balance     : Rs. %.2f%n", balance);
    }
}
```

Program 3: bank\Transaction.java

```
package bank;

public class Transaction {
    private static int counter = 1000;

    public static void deposit(Account acc, double amount) {
        acc.deposit(amount);
        counter++;
        System.out.printf("Txn %d : Deposited Rs. %.2f%n", counter, amount);
    }

    public static void withdraw(Account acc, double amount) {
        counter++;
        if (acc.withdraw(amount)) {
            System.out.printf("Txn %d : Withdrew Rs. %.2f%n",
                              counter, amount);
        } else {
            System.out.println("Txn " + counter + " : Failed - low balance");
        }
    }
}
```

Program 4: BankApp.java

```
import bank.Account;
import bank.Customer;
import bank.Transaction;

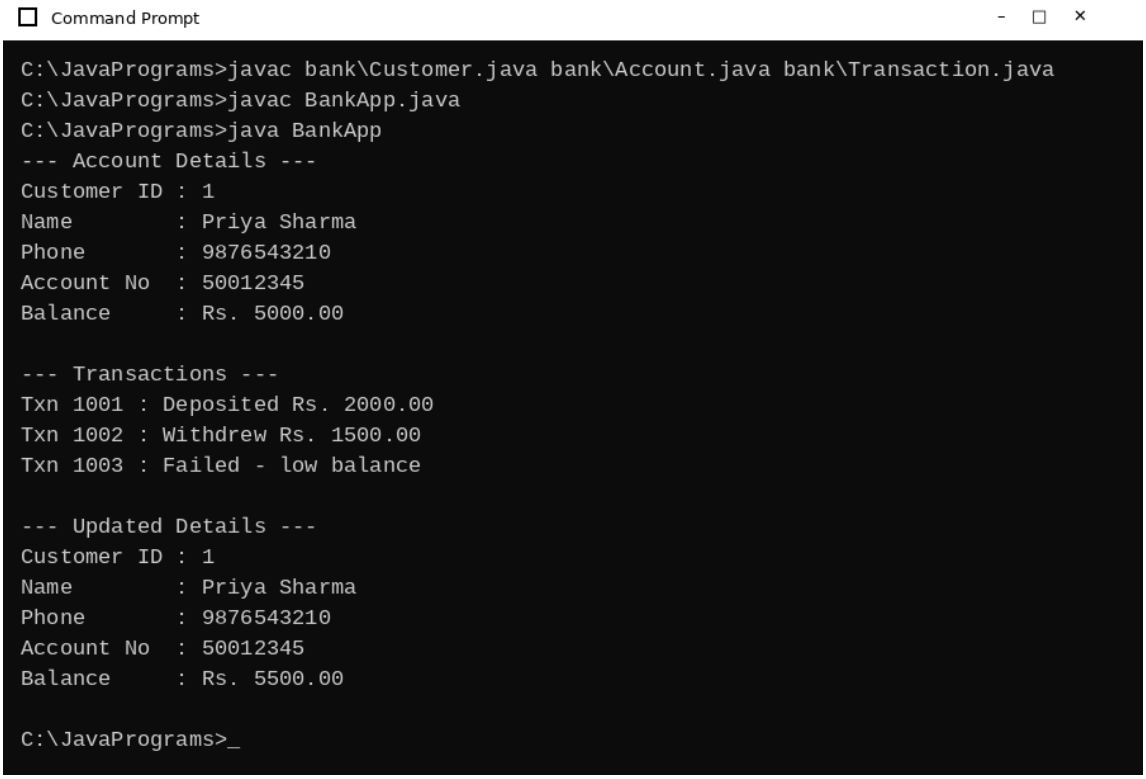
public class BankApp {
    public static void main(String[] args) {
        Customer c = new Customer(1, "Priya Sharma", "9876543210");
        Account acc = new Account(50012345L, c, 5000);

        System.out.println("--- Account Details ---");
        acc.display();

        System.out.println("\n--- Transactions ---");
        Transaction.deposit(acc, 2000);
        Transaction.withdraw(acc, 1500);
        Transaction.withdraw(acc, 10000);

        System.out.println("\n--- Updated Details ---");
        acc.display();
    }
}
```

Compilation and Output:



Experiment 6: Defining and Implementing an Interface

Aim: To create an interface and implement its methods in a class.

Problem Statement:

- Create an interface Shape containing a method area(). Implement the interface using Circle and Rectangle classes.

Program: ShapeDemo.java

```
interface Shape {
    double area();
}

class Circle implements Shape {
    double radius;

    Circle(double radius) {
        this.radius = radius;
    }

    public double area() {
        return Math.PI * radius * radius;
    }
}

class Rectangle implements Shape {
    double length, width;

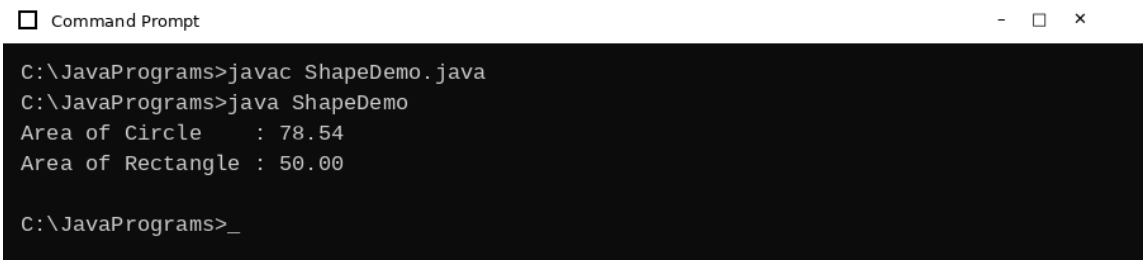
    Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    public double area() {
        return length * width;
    }
}

public class ShapeDemo {
    public static void main(String[] args) {
        Shape s1 = new Circle(5);
        Shape s2 = new Rectangle(10, 5);

        System.out.printf("Area of Circle    : %.2f%n", s1.area());
        System.out.printf("Area of Rectangle : %.2f%n", s2.area());
    }
}
```

Compilation and Output:



Experiment 7: Implementing Multiple Interfaces

Aim: To demonstrate how a class can implement multiple interfaces.

Problem Statement:

- Create two interfaces Printable (with print()) and Showable (with show()).
- Create a class Demo that implements both interfaces.

Program: Demo.java

```
interface Printable {
    void print();
}

interface Showable {
    void show();
}

public class Demo implements Printable, Showable {
    public void print() {
        System.out.println("Printing...");
    }

    public void show() {
        System.out.println("Showing...");
    }

    public static void main(String[] args) {
        Demo d = new Demo();
        d.print();
        d.show();
    }
}
```

Compilation and Output:



Experiment 8: Interface-Based Polymorphism

Aim: To demonstrate polymorphism using an interface reference.

Problem Statement:

- Create an interface Vehicle with a method start(). Implement it using Car and Bike.
- Use a Vehicle reference and assign objects of both Car and Bike to it.

Program: VehicleDemo.java

```
interface Vehicle {
    void start();
}

class Car implements Vehicle {
    public void start() {
        System.out.println("Car starts with a key.");
    }
}

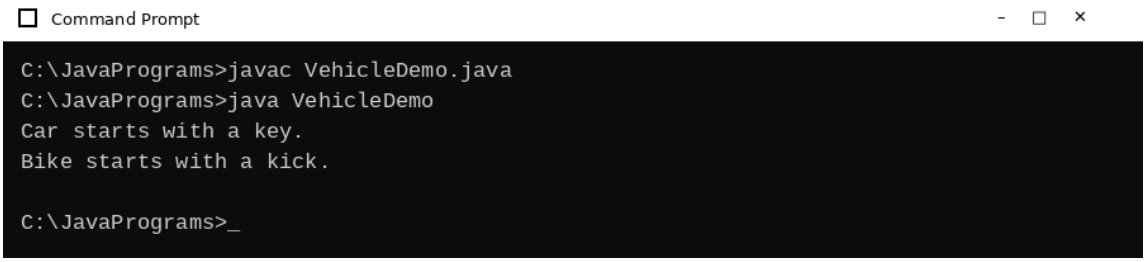
class Bike implements Vehicle {
    public void start() {
        System.out.println("Bike starts with a kick.");
    }
}

public class VehicleDemo {
    public static void main(String[] args) {
        Vehicle v;

        v = new Car();
        v.start();

        v = new Bike();
        v.start();
    }
}
```

Compilation and Output:



Experiment 9: Variables in Interfaces

Aim: To demonstrate variables declared inside an interface.

Tasks:

- Declare variables in an interface.
- Access them using the interface name.
- Demonstrate that interface variables are constants.

Program 1: Constants.java

```
public interface Constants {
    int MAX_MARKS = 100;        // public static final by default
    double PI = 3.14159;
}
```

Program 2: ConstantsDemo.java

```
public class ConstantsDemo implements Constants {
    public static void main(String[] args) {
        int marks = 85;
        double radius = 5;

        System.out.println("Constants.MAX_MARKS : " + Constants.MAX_MARKS);
        System.out.println("Constants.PI        : " + Constants.PI);
        System.out.println("MAX_MARKS (direct)  : " + MAX_MARKS);

        double percent = marks * 100.0 / MAX_MARKS;
        System.out.println("Percentage          : " + percent + "%");
        System.out.printf("Circle area (r = 5) : %.2f%n", PI * radius * radius);
    }
}
```

Program 3: ConstantChangeTest.java

```
public class ConstantChangeTest implements Constants {
    public static void main(String[] args) {
        MAX_MARKS = 200;     // trying to change a constant
        System.out.println(MAX_MARKS);
    }
}
```

Compilation and Output:



Experiment 10: Nested Interfaces

Aim: To demonstrate the concept of nested interfaces.

Problem Statement:

- Create an outer class University containing a nested interface Department.
- Implement the nested interface in another class and demonstrate its functionality.

Program 1: University.java

```
public class University {
    public interface Department {
        void showDepartment();
        void showHOD();
    }
}
```

Program 2: UniversityDemo.java

```
class ComputerScience implements University.Department {
    public void showDepartment() {
        System.out.println("Department : Computer Science");
    }

    public void showHOD() {
        System.out.println("HOD        : Dr. Rao");
    }
}

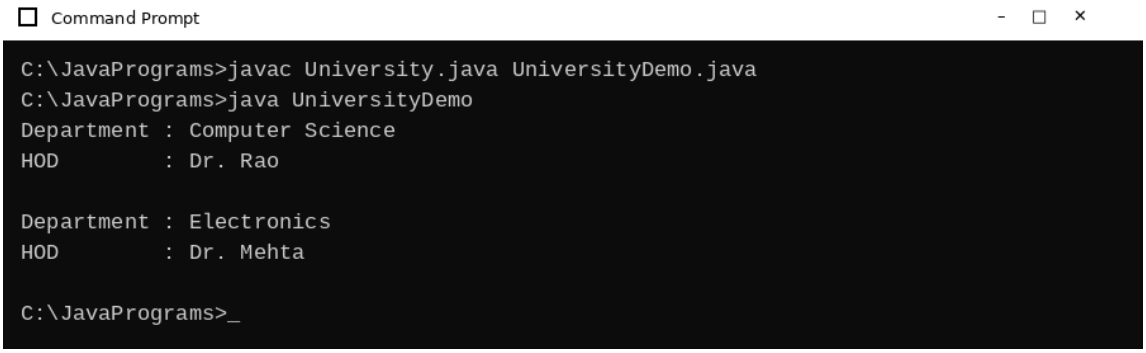
class Electronics implements University.Department {
    public void showDepartment() {
        System.out.println("Department : Electronics");
    }

    public void showHOD() {
        System.out.println("HOD        : Dr. Mehta");
    }
}

public class UniversityDemo {
    public static void main(String[] args) {
        University.Department d1 = new ComputerScience();
        University.Department d2 = new Electronics();

        d1.showDepartment();
        d1.showHOD();
        System.out.println();
        d2.showDepartment();
        d2.showHOD();
    }
}
```

Compilation and Output:



Experiment 11: Interface Inheritance

Aim: To demonstrate inheritance between interfaces.

Problem Statement:

- Create interface Animal with eat() and interface Dog (extends Animal) with bark().
- Implement both methods in the class Labrador.

Program: InterfaceInheritance.java

```
interface Animal {
    void eat();
}

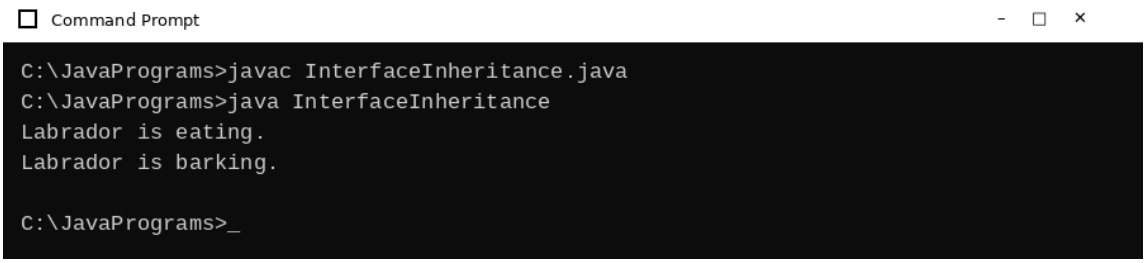
interface Dog extends Animal {
    void bark();
}

class Labrador implements Dog {
    public void eat() {
        System.out.println("Labrador is eating.");
    }

    public void bark() {
        System.out.println("Labrador is barking.");
    }
}

public class InterfaceInheritance {
    public static void main(String[] args) {
        Labrador l = new Labrador();
        l.eat();
        l.bark();
    }
}
```

Compilation and Output:



Experiment 12: Real-World Application Using Interfaces

Aim: To develop a Java application using interfaces.

Problem Statement:

- Develop a Payment Processing System using an interface Payment with the method void pay(double amount).
- Implement it using CreditCardPayment, UPIPayment and NetBankingPayment.
- The main program should allow the user to select a payment method and process the payment.

Program: PaymentApp.java

```
import java.util.Scanner;

interface Payment {
    void pay(double amount);
}

class CreditCardPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Processing credit card payment...");
        System.out.printf("Rs. %.2f paid using Credit Card.%n", amount);
    }
}

class UPIPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Processing UPI payment...");
        System.out.printf("Rs. %.2f paid using UPI.%n", amount);
    }
}

class NetBankingPayment implements Payment {
    public void pay(double amount) {
        System.out.println("Processing net banking payment...");
        System.out.printf("Rs. %.2f paid using Net Banking.%n", amount);
    }
}

public class PaymentApp {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("===== Payment Processing System =====");
        System.out.println("1. Credit Card");
        System.out.println("2. UPI");
        System.out.println("3. Net Banking");
        System.out.print("Enter your choice: ");
        int choice = sc.nextInt();
        System.out.print("Enter amount: ");
        double amount = sc.nextDouble();

        Payment payment;      // interface reference (polymorphism)
        switch (choice) {
            case 1: payment = new CreditCardPayment(); break;
            case 2: payment = new UPIPayment(); break;
            case 3: payment = new NetBankingPayment(); break;
            default:
                System.out.println("Invalid choice!");
                return;
        }
        payment.pay(amount);
    }
```

```
}
```

Compilation and Output:

